CHAPTER 7: Methods and Classes

Methods and Classes: Overloading Methods, Objects as Parameters, Argument Passing, Returning Objects, Introducing Nested and Inner Classes.

Overloading Methods

Q. Explain method overloading in java

In Java, it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. When this is the case, the methods are said to be overloaded, and the process is referred to as method overloading. **Method overloading** is one of the ways that **Java supports polymorphism**.

Sample program on method overloading to **add** strings, integers and float values:

```
//Method overloading
public class MethodOverload
{
    int a1,b1;
    float f1, f2;
    String s1, s2;
    void add()
    {
        s1="VTU";
        s2=" Univaersity";
        System.out.println("String concatenation="+s1+s2);
    }
    int add(int a,int b)
    {
        a1=a;
        b1=b;
        return (a+b);
    }
    void add(float f1,float f2)
    {
        this.f1=f1;
        this.f2=f2;
        System.out.println("Float sum="+(f1+f2));
    }
```

```
    public static void main(String args[])
    {
        MethodOverload m1=new MethodOverload();
        m1.add();
        System.out.println("Integer sum="+m1.add(10,12));
        m1.add(2.3f,5.2f);
    }
}
```

The above class definition contains overloaded method **add()**; three versions of **add()** method is used.

- **void add():** method concatenates two strings s1 and s1 and prints them.
- **int add(int a, int b):** adds two **integers** a & b and returns the sum to the main() function
- **void add(float f1, float f2):** adds two **floating** integers f1 & f2 and prints the sum.

Second sample program on method overloading which computes the **volume** of a sphere, cylinder and cuboid.

```
//Method overloading
public class OverloadMethods
{
    int radius;
    int  length, breadth, height;
    float volume, pi=3.142f;

    void volume(int r)
    {
        radius=r;
        volume=(4/3)*pi*radius*radius*radius;
        System.out.println("Volume of a sphere="+volume);
    }
    void volume(int h, int l)
    {
        height=h;
        length=l;
        volume=pi*radius*radius*height;
        System.out.println("Volume of a cylinder="+volume);
    }
    void volume()
    {
        breadth=9;
        volume=length*breadth*height;
        System.out.println("Volume of a cuboid="+volume);
    }
    public static void main(String args[])
    {
        OverloadMethods obj1=new OverloadMethods();
        obj1.volume(2);
        obj1.volume(2,3);
        obj1.volume();
    }
}
```

Output:

```
Volume of a sphere=25.136
Volume of a cylinder=25.136
Volume of a cuboid=54.0
```

In the above program volume() method is overloaded method. Three versions of volume() method.

- **volume(int r):** Computes the volume of a sphere.
- **volume(int h, int i)):** Computes the volume of a cylinder.
- **volume():** Computes the volume of a cuboid.

Using Objects as Parameters

Q. How do you use objects as parameters in Java? Provide a straightforward example.

Like primitive data types, **objects** can also be passed as parameters to methods in Java. When passing an **object** as a parameter to a method, a reference to the object is passed rather than a copy of the object itself. This means that any modifications made to the object within the method will have an impact on the original object. Consider the following program demonstrating the object as parameters in Java:

```java
//Passing object as parameter to constructor
public class Add
{
    int a, b;

    Add(Add A)
    {
        a = A.a;
        b = A.b;
    }
    Add(int x, int y)
    {
        a = x;
        b = y;
    }

    void sum()
    {
        int sum1 = a + b;
        System.out.println("Sum of a and b :" + sum1);
    }
```

```
    public static void main(String arg[])
    {
        Add A = new Add(15, 8);
        Add A1 = new Add(A);  //Object A as parameter
        A1.sum();

    }
}
```

Output:

```
Sum of a and b :23
```

Returning Objects

Q How do you return an object in Java?

User defined methods can return a value of any data type like ***int, char, float, double*** etc., like that a method can return **object** of class type.

Consider the sample program in which method **Test1 add()** returns an object of type **Test1** to the main function.

```
//Returning Objects
public class Test1
{
    int a;
    Test1(int a)
    {
        this.a=a;
    }
    Test1 add()
    {
        Test1 t1=new Test1(a+10);
        return t1;
    }
    public static void main(String args[])
    {
        Test1 t2=new Test1(2);
        Test1 t3=t2.add();
```

```
        System.out.println("t2.a="+t2.a);
        System.out.println("t3.a="+t3.a);
    }
}
```

Recursion

Q. Explain Recursion in java

Java supports recursion. Recursion is the process of defining something in terms of itself. As it relates to Java programming, recursion is the attribute that allows a method to call itself. A method that calls itself

is said to be **recursive**.

```
// A simple example of recursion.
public class Factorial
{
     // this is a recursive method
     int fact(int n)
     {
         int result;
         if(n==1) return 1;
         result = fact(n-1) * n;
         return result;
     }
     public static void main(String args[])
     {
         Factorial f = new Factorial();
         System.out.println("Factorial of 3 is " + f.fact(3));
         System.out.println("Factorial of 4 is " + f.fact(4));
         System.out.println("Factorial of 5 is " + f.fact(5));
     }
}
```

Output:

```
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

Introducing Access Control

Q. Explain access modifiers in Java?

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are **four types** of Java access modifiers:

1. **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
2. **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

```java
public class AccessTest
{
    int a;              // default access
    public int b;       // public access
    private int c;      // private access

    // methods to access c
    void set_c(int i)
    {
        c = i;
    }
    int get_c()
    {
        return c;
    }
    public static void main(String args[])
    {
        AccessTest ob = new AccessTest();
        // These are OK, a and b may be accessed directly
        ob.a = 10;
        ob.b = 20;
        // This is not OK and will cause an error
        // ob.c = 100; // Error!
        // You must access c through its methods
        ob.set_c(100); // OK
        System.out.println("a, b, and c: " + ob.a + " " +
                           ob.b + " " + ob.get_c());
    }
}
```

Output:

```
a, b, and c: 10 20 100
```

Introducing Nested and Inner Classes

Q. Explain Nested and Inner Classes

- It is possible to define a class within another class; such classes are known as **nested classes**.
- The scope of a nested class is bounded by the scope of its enclosing class. Thus, if class B is defined within class A, then B does not exist independently of A.
- A nested class has access to the members, including private members, of the class in which it is nested. However, the enclosing class does not have access to the members of the nested class. A nested class that is declared directly within its enclosing class scope is a member of its enclosing class. It is also possible to declare a nested class that is local to a block.
- There are two types of nested classes: **static** and **non-static.**

- A static nested class is one that has the static modifier applied. Because it is static, it must access the non-static members of its enclosing class through an object. That is, it cannot refer to non-static members of its enclosing class directly. Because of this restriction, static nested classes are seldom used.
- The most important type of nested class is the **inner class**. An inner class is a non-static nested class. It has access to all of the variables and methods of its outer class and may refer to them directly in the same way that other non-static members of the outer class do.
- The following program illustrates how to define and use an inner class. The class named Outer has one instance variable named **outer_x,** one instance method named **test( ),** and defines one inner class called Inner.

```
// Demonstrate an inner class.
public class Outer
{
    int outer_x = 100;
    void test()
    {
        Inner in = new Inner();
        in.display();
    }
    // this is an inner class
    class Inner
    {
        void display()
        {
            System.out.println("display: outer_x = " + outer_x);
        }
    }
    public static void main(String args[])
     {
         Outer Ot= new Outer();
         Ot.test();
     }
}
```

Output:

```
display: outer_x = 100
```